# *Chapter 1: Introduction to Information Theory*

*Solutions and comments by C Jones*

## *1.2*

Consider the binary symmetric channel with noise level $f$. By definition, transmitting a single bit has probability of error

$$p_b = f.$$

Now, consider the three-bit repetition code $R_3$. Transmitting a block results in an error if two or three bits are flipped

$$p_b = f^3 + 3f^2(1-f) = 3f^2 - 2f^3.$$

Where each of the terms in the sum may be derived from the Binomial probability mass function $X \sim B(3, f)$.

## *1.3*

For the N-bit repetition code $R_N$, let $X \sim B(N, f)$ be the number of bits flipped. From the definition of the Binomial distribution, we have

$$p(x) = C_x^N f^x (1-f)^{(N-x)}, \quad \text{where}$$
$$C_x^N = \frac{N!}{x!(N-x)!}$$

Note that this code is only reasonable for odd N, else a majority vote decoding algorithm may end up with ties.

Given the majority vote decoder, this code clearly fails iff $> \frac{N}{2}$ bit are flipped, so we can compute the probability of failure by summing the probability mass assigned to events $p((N+1)/2) \ldots p(N)$

$$p_b = \sum_{x=(N+1)/2}^{N} p(x) = \sum_{x=(N+1)/2}^{N} C_x^N f^x (1-f)^{(N-x)},$$

where $(N+1)/2$ is equivalent to `ceil(N/2)` for odd N. That is to say, the smallest integer $\geq N/2$. Note that we can express the same result through the CDF, i.e. $p_b = P(N) - P((N+2)/2)$.

We wish to derive an approximation of this formula which admits a closed form solution for computing N. The sum is clearly dominated by the $x = (N+1)/2$ term when $f$ is small, so we will throw away other terms in our simplification. We will also forget about the ceiling rounding trick and use $x = N/2$ everywhere, giving

$$p_b \approx C_{N/2}^N (f - f^2)^{N/2}.$$

Being a computer scientist, I was initially surprised that we're not done here – surely we can just use a discrete optimisation approach to compute N for any desired $p_b$ and $f$! I do that below as an extension, however Mackay's approach of using approximations to find an analytical solution is particularly enlightening.

We can now use Stirling's approximation $x! \approx x^x e^{-x}\sqrt{2\pi x}$ to tidy up the $C^N_{N/2}$ term

$$C^N_{N/2} \approx 2^{NH_2\left(\frac{N}{2N}\right)}, \quad \text{where}$$

$$H_2(x) = x\log\left(\frac{1}{x}\right) + (1-x)\log\left(\frac{1}{1-x}\right) \quad \text{(binary entropy function).}$$

As it happens, the binary entropy of $1/2$ is exactly 1, giving us the somewhat intuitive notion that the 'middle' binomial coefficient is upper bounded by $2^N$. This allows us to simplify even further to

The $C^N_{N/2} \leq 2^N$ bound is particularly intuitive once we notice that the sum of all binomial coefficients is *exactly* $2^N$! That is, $\sum_{i=0\ldots N} C^N_i = 2^N$.

$$p_b \approx (4f - 4f^2)^{N/2}.$$

We may now solve analytically to find N such that the probability of error is $10^{-15}$ when $f = 0.1$:

$$2\log(10^{-15}) = N\log(0.36)$$

$$\implies N = 67.6.$$

Question 1.3 is solved now, but I actually think we can take it further with a few more instructive steps.

Interestingly, we can prove that this is an upper bound on $N$, not simply an approximation. This is not immediately obvious; for our estimate to be an upper bound, our approximation of $p_b$ must be $\geq$ the true value. We made three simplifying assumptions to $p_b$, which we will go through individually:

- First, we used $(N+1)/2 \approx N/2$. It's straightforward to see how this strictly increases the estimate.
- Second, we used $C^N_{N/2} \approx 2^N$. Again, since $\sum_{i=1\ldots N} C^N_i = 2^N$, this must increase our estimate.
- Finally, we threw away all terms in the sum other than the 'middle' $n = N/2$ term. Since all terms are positive (remember that it's a probability mass function), this actually deflates our estimate of $p_b$!

This last point isn't encouraging, however, we can prove that the underestimation caused by throwing away these terms is always more than offset by the overestimation caused by Stirling's approximation. We'll begin by demonstrating that, for $f < 0.5$ and $n \geq N/2$, all other terms may be bounded by the middle term

$$f^n(1-f)^{(N-n)} \leq f^{N/2}(1-f)^{N/2} = (f-f^2)^{N/2}, \quad \text{thus}$$

$$\sum_{n=(N+1)/2}^{N} C^N_n f^n(1-f)^{(N-n)} \leq \sum_{n=0}^{N} C^N_n (f-f^2)^{N/2} = 2^N(f-f^2)^{N/2},$$

which finally gives us

$$\sum_{n=(N+1)/2}^{N} C_n^N f^n (1-f)^{(N-n)} \leq (4f - 4f^2)^{N/2}.$$

The fact that our estimate is a true upper bound is interesting for two reasons. First, and somewhat remarkably, this exactly the same as the result we would obtain if we used moment-generating functions to derive the Chernoff (Bhattacharyya) bound. I won't prove this here.

Second, now that we have a good upper bound, solving the discrete optimisation problem to compute the exact value becomes even easier. Without a good bound, the best method would be to perform an exponential search to establish loose bounds on $N$, then a binary search to find the true value ($\mathcal{O}(\log N)$ time overall). Now, we can simply perform a local linear search starting at $N = 69$, which is likely to be faster in practice.

Running this search gives us $N = 63$ as the minimum length repetition code to reduce the probability of error below $10^{-15}$. Below, I give a contour visualisation of the probability of error across a range of configurations.

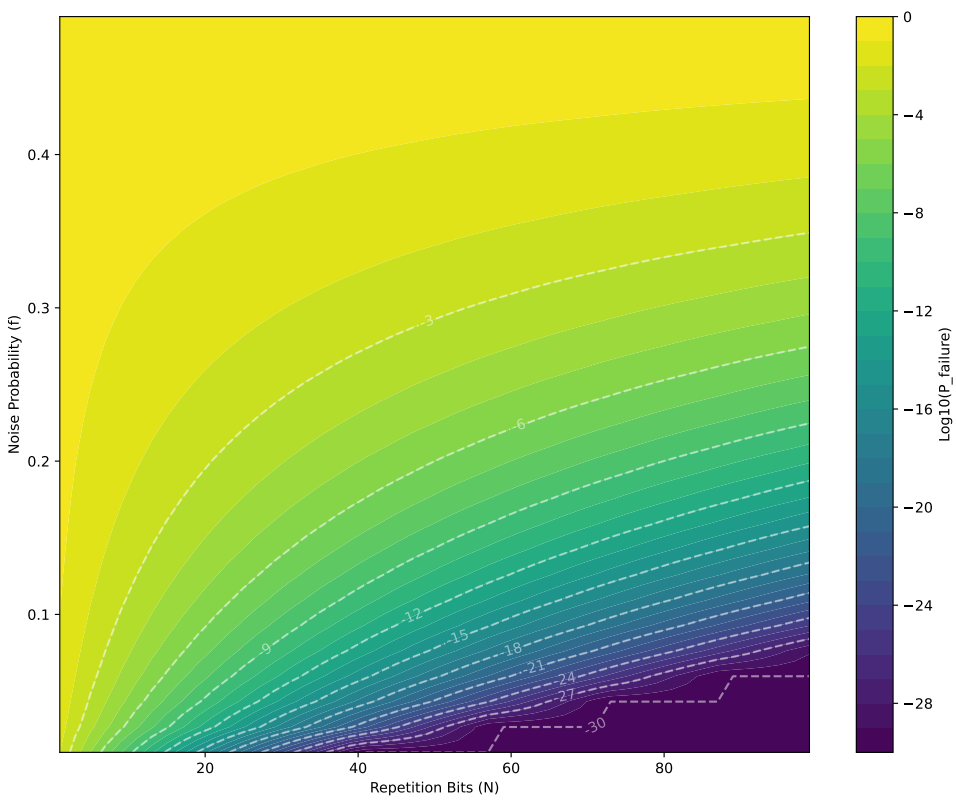


Figure 1: Probability of failure as a function of $f, N$. Contours represent isolines for log-probability of failure.